

Week 4: scipy.stats, tests and simulations

Python syntax and code patterns. Keep it next to you while you work.

The import

```
from scipy import stats
```

The three tests

```
# two independent groups, the default choice
stats.ttest_ind(a, b, equal_var=False)    # Welch
stats.ttest_ind(a, b, equal_var=True)     # Student

# the same units measured twice
stats.ttest_rel(before, after)

# one group against a fixed value
stats.ttest_1samp(x, popmean=0.5)
stats.ttest_1samp(x, popmean=0.5,
                  alternative="greater")   # one-sided
```

Reading the result

```
res = stats.ttest_ind(a, b, equal_var=False)
res.statistic    # t
res.pvalue       # p, two-sided unless you say otherwise
res.df          # degrees of freedom

print(f"t = {res.statistic:.3f}, df = {res.df:.1f}, "
      f"p = {res.pvalue:.4g}")
```

Deciding

```
decision = "reject" if res.pvalue < alpha else "do not reject"

for null_value in [0.5, 0.55]:
    res = stats.ttest_1samp(x, popmean=null_value)
    print(f"H0 mean = {null_value}: p = {res.pvalue:.4g}")

alpha_corrected = 0.05 / n_tests    # Bonferroni
```

The statistic by hand

```
n = len(x)
se = np.std(x, ddof=1) / np.sqrt(n)
t = np.mean(x) / se
df = n - 1
p = 2 * stats.t.sf(np.abs(t), df)    # two-sided
```

Two groups by hand

```
n_a, n_b = len(a), len(b)
pooled_var = ((n_a - 1) * np.var(a, ddof=1)
              + (n_b - 1) * np.var(b, ddof=1)) / (n_a + n_b - 2)
pooled_sd = np.sqrt(pooled_var)
se = pooled_sd * np.sqrt(1 / n_a + 1 / n_b)
t = (np.mean(a) - np.mean(b)) / se
df = n_a + n_b - 2
```

Effect size and interval

```
d = (np.mean(a) - np.mean(b)) / pooled_sd    # Cohen's d

critical = stats.t.ppf(0.975, df)             # two-sided 95%
margin = critical * se
low, high = difference - margin, difference + margin
```

The distributions

stats.t.sf(x, df)	upper tail area, double it for two-sided
stats.t.ppf(q, df)	the inverse, gives critical values
stats.t.pdf(x, df)	the curve, for plotting
stats.norm.sf / ppf / pdf	the same for the Normal
stats.levene(a, b)	do the two spreads differ?

Pattern: power by simulation

```
def simulate_power(d, n, rng, repeats=2000, alpha=0.05):
    hits = 0
    for _ in range(repeats):
        a = rng.normal(0, 1, size=n)
        b = rng.normal(d, 1, size=n)
        if stats.ttest_ind(a, b, equal_var=False).pvalue < alpha:
            hits += 1
    return hits / repeats

sizes = [10, 20, 30, 40, 60]
curve = [simulate_power(d, n, rng) for n in sizes]
enough = [n for n, p in zip(sizes, curve) if p >= 0.8]
```

Pattern: a function returning a report

```
def compare_groups(a, b, alpha=0.05):
    equal = stats.levene(a, b).pvalue >= alpha
    res = stats.ttest_ind(a, b, equal_var=equal)
    low, high = difference_ci(a, b)
    return {"test": "Student" if equal else "Welch",
            "t": round(float(res.statistic), 3),
            "p": round(float(res.pvalue), 4),
            "ci95": (round(float(low), 3),
                     round(float(high), 3))}
```

Pattern: from table to test

```
valid = trained[trained["rt_s"].between(0.08, 2.0)]
per_mouse = (valid
             .groupby(["cohort", "subject_id"])["rt_s"]
             .median()
             .reset_index())

cshl = per_mouse.loc[per_mouse["cohort"] == "CSHL-WT",
                    "rt_s"].to_numpy()
nyu = per_mouse.loc[per_mouse["cohort"] == "NYU-WT",
                    "rt_s"].to_numpy()
res = stats.ttest_ind(cshl, nyu, equal_var=False)
```

One number per mouse, never one per trial.

Pattern: several comparisons

```
comparisons = [("genotype", wt, asd),
               ("laboratory", cshl, nyu),
               ("sex", male, female)]

for name, a, b in comparisons:
    res = stats.ttest_ind(a, b, equal_var=False)
    print(f"{name:<12} t = {res.statistic:6.3f} "
          f"p = {res.pvalue:.4f}")
```

Drawing a null distribution

```
fig, ax = plt.subplots()
ax.hist(null_values, bins=40, alpha=0.7)
ax.axvline(observed, color="red", lw=2, label="observed")
ax.legend(frameon=False)

np.isclose(ours, theirs)    # our answer against scipy's
```

Drawing a power curve

```
fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(sizes, curve, "o-", lw=2, ms=6, label="observed d")
ax.axhline(0.8, color="black", lw=1.5, ls="--",
           label="80% power")
ax.set_xlabel("mice per group"); ax.set_ylabel("power")
ax.set_ylim(0, 1); ax.legend(frameon=False)
```